

# Лабораторная работа 1. Концепция и спецификация проекта

---

**Цель:** определить концепцию проекта, собрать и структурировать требования.

**Репозиторий и коммиты:**

- Все документы и код курса ведутся в **одном репозитории** (Git) на протяжении всех лабораторных работ.
- Репозиторий не обязательно публичный — допустим локальный, но **обязательно с сохранением истории изменений**.
- После выполнения лабораторной работы сделайте **как минимум один коммит**: документация работы (отчёт) и материалы (презентация).

**Содержание:**

1. Формирование команды из 2 человек или индивидуальная работа. *Для команды объём работы увеличивается.*
2. Выбор идеи проекта (согласование с преподавателем).
3. Описание проблемы и целевой аудитории.
4. Аналитический обзор аналогов: поиск существующих решений и сравнение в табличной форме по объективным, измеримым характеристикам (функциональность, производительность, поддерживаемые платформы, интеграции и т.п.) — **10 характеристик**.
  - *Для одного студента:* не менее **3 аналогов**.
  - *Для команды:* не менее **5 аналогов**.
5. Формулирование User Stories — они описывают потребность с точки зрения пользователя и отвечают на вопрос «зачем».

Формат: «Я, как [роль], хочу [действие], чтобы [ценность]».

- *Для одного студента:* не менее **8–10 User Stories**.
- *Для пары:* не менее **15–18 User Stories**.

Каждая User Story должна:

- описывать потребность пользователя, а не техническую реализацию;
- быть достаточно независимой, чтобы её можно было обсудить отдельно;

- служить основой для вывода функциональных требований (см. п. 6).

## 6. Вывод функциональных требований (FR) из User Stories.

На основании сформулированных User Stories выведите функциональные требования — они описывают, что именно должна делать система, чтобы реализовать эту историю. Одна User Story может порождать одно или несколько FR (если история сложная) или, в редких случаях, одно FR может удовлетворять нескольким User Story.

Каждое FR должно:

- иметь уникальный идентификатор (FR-XX);
- быть проверяемым (Acceptance Criteria);
- иметь трассировку к своей User Story (указывать, из какой истории оно выведено).

Рекомендуемая доля FR: **75–85%** от общего числа требований.

- *Для одного студента:* не менее **11–13** FR.
- *Для пары:* не менее **19–21** FR.

## 7. Нефункциональные требования (NFR) — какими свойствами (производительность, безопасность, удобство использования, надёжность, масштабируемость) система должна обладать.

Нефункциональные требования должны быть **измеримыми**. Вместо «система должна быть быстрой» писать «время отклика API при 100 параллельных запросах не превышает 300 мс». Это позволит в дальнейшем объективно проверить их выполнение (валидация требований).

Рекомендуемая доля: **15–25%** от общего числа требований.

- *Для одного студента:* не менее **2–4** NFR.
- *Для пары:* не менее **4–6** NFR.

## 8. Приоритизация требований (MoSCoW — см. ниже).

## 9. Использование агента для структурирования и проверки полноты требований. В отчёте обязательно фиксируются примеры промптов и анализ полученных ответов (что агент предложил, что скорректировано и почему). Отдельно отметьте, помог ли агент при выводе FR из User Stories.

## 10. Планирование валидации требований: для каждого требования определить способ проверки (демонстрация на прототипе, тест, экспертная оценка) и представить его в отчёте (пример см. ниже).

11. Предварительная оценка рисков (технических, организационных, рыночных) — не менее 3–5 пунктов.

12. Создание питч-презентации идеи: проблема → решение → целевая аудитория → команда. Формат: до 3 минут, PDF или слайды.

При распределении требований по категориям MoSCoW рекомендуется придерживаться следующего целевого соотношения (от общего числа требований):

| Категория   | Доля   | Смысл                                                                           |
|-------------|--------|---------------------------------------------------------------------------------|
| Must-have   | 40–50% | Без этого функционала продукт теряет смысл (MVP-ядро)                           |
| Should-have | 25–30% | Важно, но при нехватке времени можно реализовать упрощённую версию или отложить |
| Could-have  | 15–20% | Желательно, но не критично; реализуется в последнюю очередь                     |
| Won't-have  | 5–10%  | Осознанно исключено из текущей версии (чтобы чётко очертить границы)            |

Примеры для разных объёмов:

- Для одного студента (13–17 требований: 11–13 FR + 2–4 NFR):
  - Must-have: 6–8
  - Should-have: 4–5
  - Could-have: 2–3
  - Won't-have: 1–2
- Для пары (23–27 требований: 19–21 FR + 4–6 NFR):
  - Must-have: 10–13
  - Should-have: 6–8
  - Could-have: 4–5
  - Won't-have: 2–3

Пример оформления спецификации требований:

Спецификация оформляется в три связанных блока.

## 1. User Stories

| ID    | User Story                                                                                                           | Приоритет (MoSCoW) |
|-------|----------------------------------------------------------------------------------------------------------------------|--------------------|
| US-01 | Я, как пользователь, хочу авторизоваться по email и паролю, чтобы получить доступ к личному кабинету.                | Must               |
| US-02 | Я, как менеджер проекта, хочу видеть список задач с фильтром по статусу, чтобы быстро находить незавершённые задачи. | Should             |

## 2. Функциональные требования (FR), выведенные из User Stories

| ID    | Из какой US | Формулировка требования                                   | Acceptance Criteria                                                                                                                                         | Приоритет (MoSCoW) |
|-------|-------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| FR-01 | US-01       | Система должна выполнять вход по email и паролю.          | 1. Поле email принимает валидный формат. 2. При неверном пароле выводится ошибка «Неверные учётные данные». 3. После успешного входа — редирект на главную. | Must               |
| FR-02 | US-02       | Система должна предоставлять фильтрацию задач по статусу. | 1. Фильтр применяется за < 1 сек. 2. Отображаются только задачи с выбранным статусом. 3. При сбросе фильтра показываются все задачи.                        | Should             |

## 3. Нефункциональные требования (NFR)

| ID     | Формулировка требования                                          | Acceptance Criteria                                                        | Приоритет (MoSCoW) |
|--------|------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------|
| NFR-01 | Система должна поддерживать до 1000 одновременных пользователей. | Load-тест: время ответа API при 1000 RPS $\leq$ 500 мс.                    | Must               |
| NFR-02 | Пароли пользователей должны храниться в хешированном виде.       | Используется bcrypt (cost 12). В коде и логах нет паролей в открытом виде. | Must               |

Пример оформления матрицы валидации:

| ID требования          | Способ проверки                   | Ожидаемый результат                |
|------------------------|-----------------------------------|------------------------------------|
| FR-01<br>(Авторизация) | Автоматизированный тест           | Неверный пароль возвращает код 401 |
| FR-02 (Фильтр задач)   | Демонстрация на прототипе         | Список обновляется за < 1 сек.     |
| NFR-01 (Нагрузка)      | Нагрузочное тестирование (JMeter) | Время ответа < 500 мс при 1000 RPS |

**Результат лабораторной работы:** документ с описанием проекта, аналитическим обзором аналогов (с таблицей сравнения), спецификацией требований (в трёх связанных блоках: User Stories → FR → NFR, с учётом дифференциации по количеству), планом валидации, оценкой рисков и отчётом о работе с агентом (в т.ч. о помощи агента при выводе FR из User Stories), а также питч-презентация.

**Структура отчёта:**

- **Описание проблемы и целевой аудитории** — какую боль решает проект и кому он нужен.
- **Аналитический обзор аналогов** — таблица сравнения (объективные и измеримые характеристики), вывод о месте собственного продукта.
- **Спецификация требований** — три связанных блока: User Stories (US-XX с приоритетом MoSCoW) → функциональные требования (FR-XX с трассировкой к User Story и Acceptance Criteria) → нефункциональные требования (NFR-XX с измеримыми критериями).

- **План валидации требований** — матрица «требование → способ проверки → ожидаемый результат».
- **Оценка рисков** — перечень и описание.
- **Отчёт о работе с агентом** — какие запросы делались, что получилось, что пришлось скорректировать, помог ли агент при выводе FR из User Stories.

#### Критерии оценки:

- Актуальность и реалистичность идеи.
- Качество аналитического обзора: объективность и измеримость характеристик, полнота сравнения (для пары — большее число аналогов).
- Полнота и качество User Stories (соответствие формату, независимость, ориентация на потребность).
- Корректность трассировки FR ← User Story: каждое FR выведено из истории, нет «висящих» FR без истории и историй без FR.
- Полнота и корректность FR (количество и разнообразие), качество Acceptance Criteria.
- Измеримость NFR.
- Корректность MoSCoW и матрицы валидации.
- Продуманность плана валидации.
- Качество взаимодействия с агентом (отчёт о процессе, примеры промптов, критический анализ, в т.ч. помощь агента при выводе FR из User Stories).
- Ясность и убедительность питч-презентации.
- Наличие репозитория с сохранённой историей изменений и коммита по итогам работы.